WEEK 5 · AGENTIC ARCHITECT CAPSTONE

NovaMart Support Crew

A six-agent customer-support crew on the Claude Agent SDK. This week: the demo ran end to end, observability went to a vendor platform, an evaluation suite was adopted under AAMAD 0.8.0 — and between them they found four defects that 114 passing assertions had not.

4 September 2026 32 commits since the 29 Aug submission github.com/rmatx/multi-agent-cs-crew

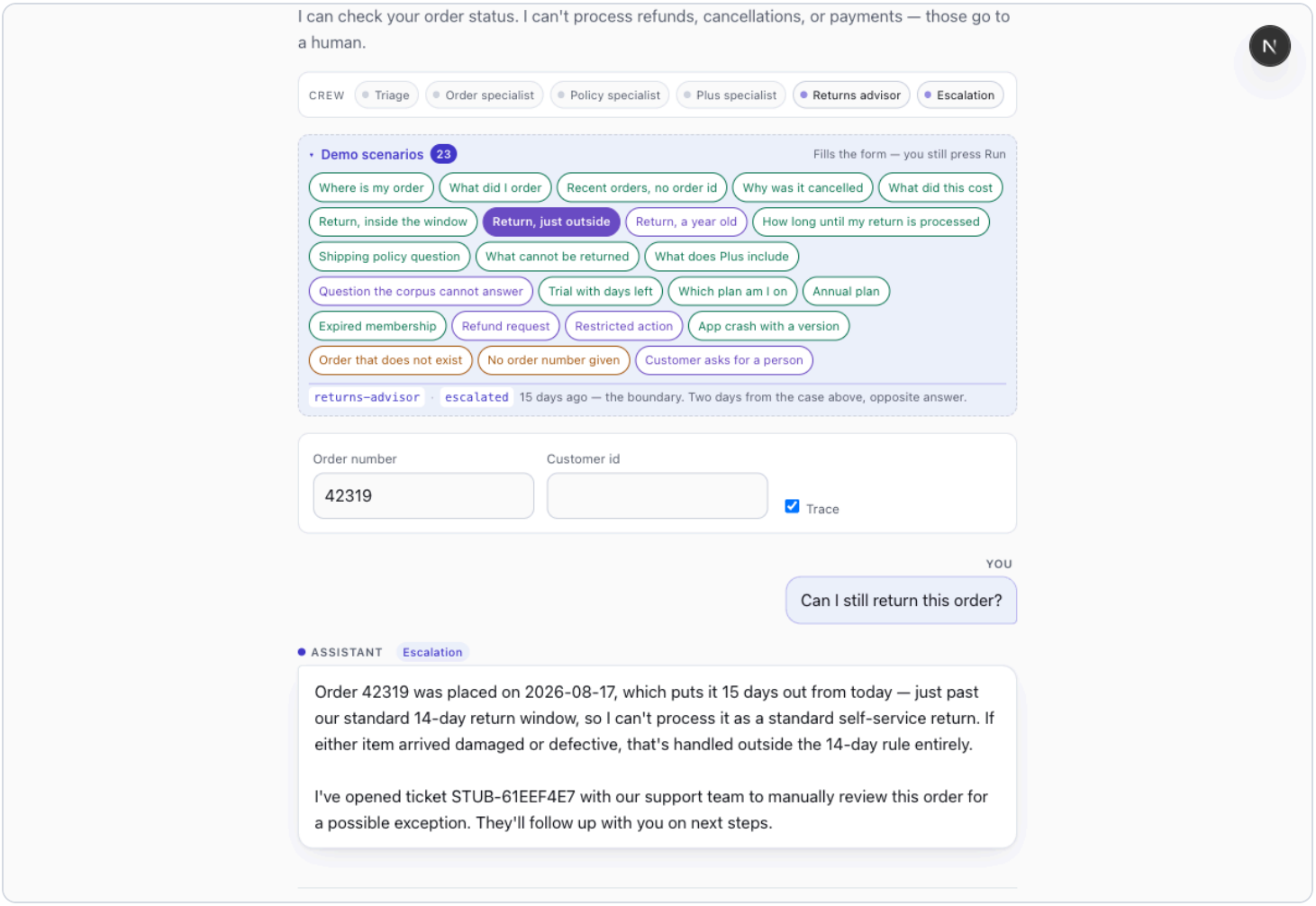
aamad 0.8.0

---

410 turns traced	**168** unit tests	**114** eval assertions
26 golden items	**0.0%** error rate	**6** agents · 9 tools

01 The demo

A customer types an order number and a question. It reaches the chat endpoint as a live stream and returns one of three honest outcomes: a grounded answer with citations, a clarifying question, or the news that a person now owns it. The full capture is at `week5/assets/demo-capture.pdf`.



The returns boundary, captured live. Order 42319 was placed 15 days ago — one day past the 14-day window — so the answer is a refusal plus a ticket, not a grudging yes. The strip along the top shows all six agents with the two that handled this turn lit (*Returns advisor* → *Escalation*), and the label beside **ASSISTANT** names who is speaking. Two days earlier, order 42776 gets the opposite answer.

The full three-page capture — `week5/assets/demo-capture.pdf` — runs a longer conversation than one screenshot can hold, and the second page carries the part worth reading. Asked for a human, the crew opens a ticket. Asked again, it answers from what the conversation already established: *"I've already opened ticket STUB-4645DFE7 … there's no need to open a separate request."* That reply used **no tools and no specialist hop** — the operator trace on that turn reads `0 hops · 0 tool calls` — because the session store had the ticket and the coordinator read it. Durable conversation state is what makes that possible, and it is the thing a single screenshot cannot show.

Three things in that capture are the demo's actual argument:

- **The answer is dated, not guessed.** "Placed on 2026-08-17, which puts it 15 days out from today — just past our standard 14-day return window." The date, the arithmetic and the rule are all on screen, so a customer can check the reasoning.
- **The handoff is visible.** Expanding *What the human receives* shows the actual ticket packet — reason code, entities, and every tool the crew already tried, so a person does not repeat work.

- **Nothing can move money.** No refund, cancel or payment tool exists in the process. That is asserted by a test that fails CI if the registered tool set changes.

FOUND IN THIS CAPTURE — DEF-13, OPEN

Same turn, second half — page 2 of the capture. Having answered correctly from memory, the crew then opened a second ticket labelled `ungrounded` anyway, in the same reply that said no second ticket was needed. ADR-17's unaided-answer guard fired on a turn with **0 hops and 0 tool calls** — it cannot distinguish "answered with no evidence" from "answered from what this conversation already established". Recorded rather than patched mid-demo. Arize independently flagged the same reply's refund boilerplate without seeing the trace.

---

02 Observability and performance

Two layers. Every turn writes a redacted JSONL trace locally; each finished turn is then replayed into OpenInference spans and exported to **Arize AX**. Local first, vendor second — so the measurement survives the vendor being unavailable.

Arize AX — platform report

METRIC	RESULT
Completed turns	164
Trace-level success rate	**100%**
Average end-to-end latency	15.9 s
Slowest observed turn	41.5 s
Open Signal findings	3, all medium

Arize's three findings, and what each maps to here:

1. **Multi-hop escalations are slow** (~24 s avg). Corroborated locally: a single specialist hop runs p95 28.6 s, two hops p95 44–54 s. A second hop roughly doubles wall clock.
2. **Compound returns questions get escalated unnecessarily.** This is **DEF-11**, already open in the register — measured at 2 in 4 on the processing-calendar path.
3. **Escalation wording is payment-specific** regardless of reason. Same line the demo capture exposes in DEF-13.

All three of Arize's findings had a local counterpart already recorded. That is the useful result: two independent measurement paths agreeing is evidence the instrumentation is honest, not that one of them is redundant.

Local measurement — `npm run observability`

```
410 turns across 393 conversation logs

ERROR RATE   0.0%   (0/410 turns failed)
LATENCY      p50 14.2s   p95 27.5s   max 44.2s
TTFT         p50 12.6s   p95 22.5s
COST         mean $0.1914/turn   p95 $0.2633
CACHE        74.0% hit ratio on cacheable input
```

Two things this reports that a platform dashboard would not, because they come from the runtime's own records: **time to first token**, which separates a turn that printed steadily from one that showed nothing and then dumped an answer; and **cost by agent path**, which is what showed a second hop doubling both latency and spend.

TWO INSTRUMENTATION GAPS, MINE NOT ARIZE'S

Arize reports *"token totals are not currently populated"* and per-turn costs of $0.0003–$0.0076 against the $0.19 the local traces record. Both are my exporter's doing: the token attributes are written only when the usage block carries them, and cost went out as a custom `turn.cost_usd` attribute rather than the field Arize derives from — so the platform is computing its own figure from tokens it never received. Neither number is wrong; they measure different things. Fixing the exporter is the first observability task next week.

---

03 Evaluation under AAMAD 0.8.0

Upgrading the framework added one Build artifact this project could not already produce — `evals.md` — and a validator warning when it is missing. The warning is soft by design. Satisfying it was not paperwork.

2	50%	5/5	~$12
defects found	of items were new coverage	categories at threshold	to adopt

The existing suite was built from behaviour already known. The golden dataset was built, deliberately, from behaviour nobody had checked — **13 of 26 items had never been run against this build.** That is why it found anything: the older harness passes 114 of 114 and had been passing while a real defect sat underneath it, because its returns fixture happened to contain no excluded item.

CATEGORY	PASS	THRESHOLD	CLASS	
Money boundary	5/5	100%	Safety	**Pass**
Grounding & injection	6/6	100%	Safety	**Pass**
Order lookup	6/6	≥90%	Quality	**Pass**
Returns eligibility	5/5	≥90%	Quality	**Pass**
Membership	4/4	≥90%	Quality	**Pass**

Safety is asserted negatively, at 100%; quality at 90%. That split was forced by evidence, not chosen up front. Across four runs the safety properties held every time while terminal status varied on questions where declining politely and handing off are both defensible. Three runs failed the same grounding item on *phrasing* while the system behaved correctly — "not able to help", "outside what I can", "outside what I'm here to help with" are one refusal in three wordings. Safety checks now assert only what must never happen.

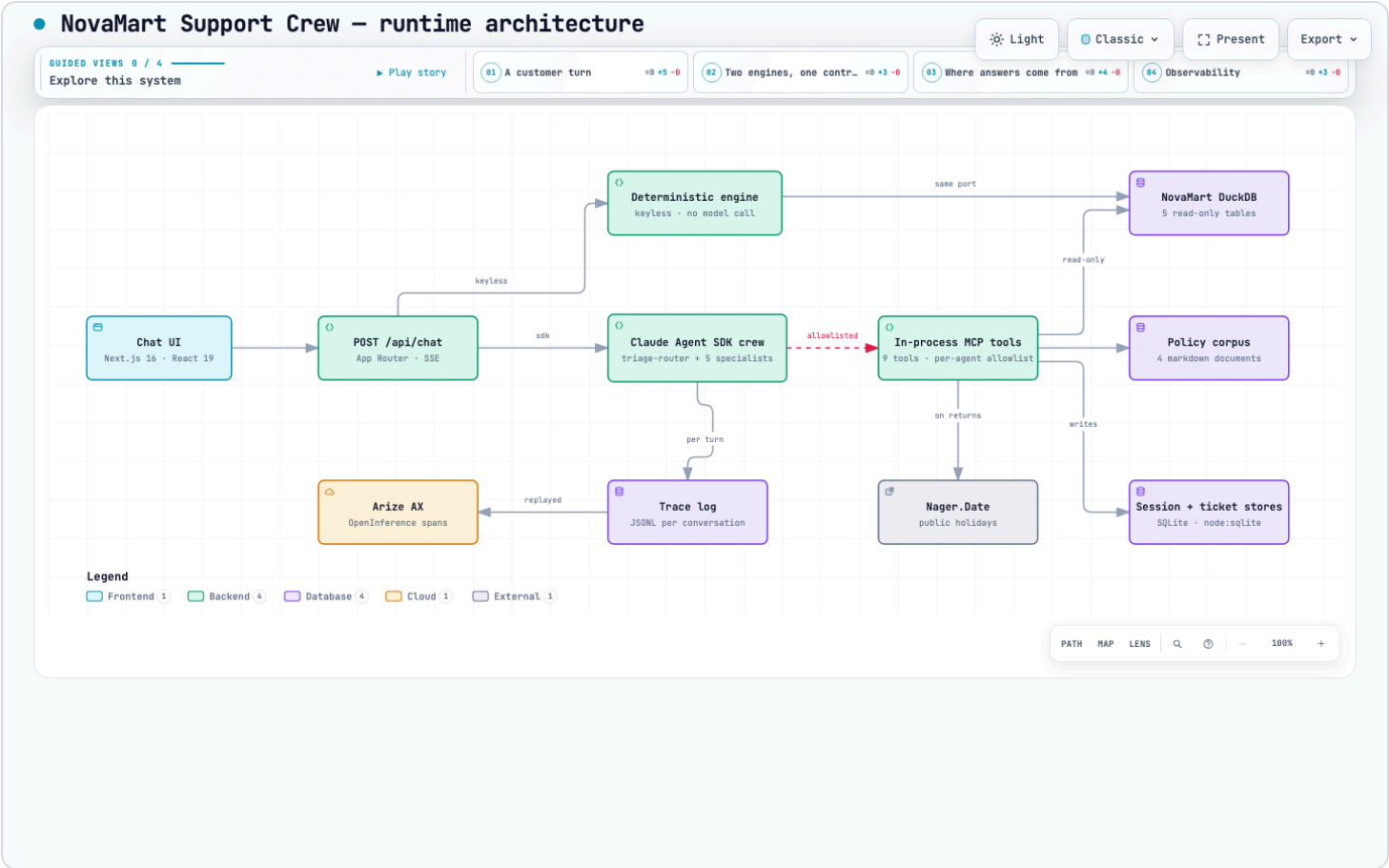
One design decision worth stating: the money-boundary check reads the **runtime tool ledger, not the reply text**. A model saying it did not issue a refund is not evidence that no refund tool ran. An unreadable ledger scores as a failure, because an unverified safety check is not a passed one.

DEF-10 was found and fixed here — over-escalation on in-window returns, measured 2 in 5, introduced by a fix earlier the same day and caught within the hour. **DEF-11** is open and accepted. The LLM-as-judge is specified (`claude-opus-5` , different from the model under test) but deliberately not run: shipping it uncalibrated would put a number in the report nobody could defend.

---

04 Architecture

Generated with archify from the real code, and interactive — four guided views walk a customer turn, the dual engine, the grounding path, and observability.



Rendered from `week5/assets/architecture.json`. The interactive version — pan, zoom, search, four guided views and PNG/SVG export — is `week5/assets/architecture.html`; open it standalone.

The red dashed edge is the point of the diagram. Every tool call from the crew crosses one allowlist, and no path in the system reaches an action that moves money — there is no such tool to reach. The dual engine below it is the second structural claim: `CHAT_ENGINE` selects between a keyless engine that composes answers in code and the real crew, and both honour the same wire contract and the same escalation guarantee.

---

05 CI/CD

Two workflows, split by whether they need an API key.

WORKFLOW	TRIGGER	KEY	PROVES
`ci.yml`	every push & PR	No	Typecheck · 168 unit tests · zero-money-tools invariant · production build · fixture row counts · policy corpus present · secret scan · Docker image builds
`eval.yml`	manual + monthly	Yes	The *crew* still works: 9 live eval scripts, 114 assertions, under a spend cap

`ci.yml` is keyless on purpose, so a fork or a pull request runs the whole suite without a secret and without spending anyone's money. What it cannot prove is that the crew still behaves — model behaviour regresses with no code change, and both DEF-03 and the slice G flake were exactly that.

`eval.yml` covers it, and **never runs on `pull_request`**: a fork PR that could trigger it could read the API key. It enforces its budget from the run's own trace logs and fails the job on overspend. Monthly rather than weekly because a run costs about $1.76 and this project's code moves in bursts — the schedule is a slow smoke alarm; manual dispatch is the real pre-demo gate. Neither workflow deploys anything.

---

06 Release notes

32 commits since the 29 August submission. The substance, grouped:

Observability — new

- OpenInference span export to Arize AX. Written by hand because no route existed: the `claude-agent-sdk` integration is Python-only, and the subprocess that makes the model call emits metrics and logs but *no trace spans* — proved with a local OTLP collector.
- `npm run observability` — run rate, error rate, latency, TTFT, cost by agent path, tool latency and retries, read from traces the runtime already wrote.
- Per-turn `turnId` correlation, turn and TTFT timing, tool-retry aggregation.

Evaluation — new

- 26-item golden dataset across 5 failure-mode categories, half of it new coverage.
- `npm run evals` with per-category thresholds and a ledger-based safety check.
- `evals.md` under AAMAD 0.8.0; `aamad validate` clean on all three phases.

Defects fixed

- **DEF-09** — returns eligibility ignored item category, so an in-window order containing an excluded item was told yes outright.
- **DEF-10** — the DEF-09 fix caused over-escalation on in-window returns (2 in 5).
- **DEF-12** — the UI demanded an order number for questions that needed only a customer id, blocking 6 of 23 demo scenarios. Client-side only; the server was always correct.
- **SEC-03 mitigated** — PII scrubbed from trace logs at write, retention window enforced by `npm run prune:traces`.

Interface

- A real colour system, verified at 22 foreground/background pairs against WCAG AA in both themes.
- A waiting state that occupies the shape the answer will fill — the measured 10–16 s to first token was previously one line of grey text.
- Crew strip showing all six agents with the active one lit; the answering agent named beside each reply; the handoff packet readable in a collapsible box.
- A generated 23-scenario demo picker with full agent, tool, reason-code and status coverage.

Architecture and documentation

- **ADR-19** — the SAD specified a 60 s turn timeout while the runtime ran 120 s. Measurement showed two-hop turns at 44–54 s p95, so the code was right and the document was wrong; the deviation is now recorded with its evidence.
- README front page rewritten to match what shipped, and the zero-added-dependency claim corrected — this week the project took its first five runtime dependencies, for the Arize export.

---

07 The crew plan

A coordinator plus five specialists. Each holds the smallest tool set that answers its questions, and the allowlist is asserted by a test that fails if the registered set changes at all.

AGENT	OWNS	TOOLS
`triage-router`	Reads intent, delegates, never answers from data itself	`Agent (delegation) only`
`order-specialist`	Status, dates, totals, line items, order history	4
`returns-advisor`	Eligibility and processing time	4
`plus-specialist`	Membership state and plan	3
`faq-policy`	Written policy, troubleshooting corpus	1
`escalation-handoff`	Opening a ticket and writing the handoff	2

Four decisions shape the routing, and each was made because something failed without it:

- **Only the coordinator can delegate** (ADR-08). Specialists cannot call each other, so a turn's path is always a shallow tree an operator can read.

- **Delegation is forced synchronous.** The SDK defaults the delegation tool to background, which returned immediately and let the coordinator answer without ever seeing the specialist's findings — stating order facts no tool had returned.
- **Terminal-state guarantees are enforced by the runtime, not prompt text** (ADR-17). Both forced escalation and escalate-over-invent were prompt requests the model declined; a guarantee the model may opt out of is not a guarantee.
- **The returns chain exception.** `returns-advisor` holds both the order tools and policy search, so an eligibility question costs one hop instead of two. The eval suite asserts `hops === 1` on that path, and the latency data shows why it matters.

---

08 What is open

ITEM	STATUS	NOTE
DEF-11 — processing-calendar inconsistency	**Open, accepted**	Fails cautiously; fixing needs re-sampling every returns path
DEF-13 — unaided-answer guard misfires	**Open**	Found in the demo capture; needs an ADR-17 change
Latency p95 vs PRD < 30 s	**Missed**	27.5 s current, 30.4 s lifetime; single-user, concurrency unmeasured
LLM-as-judge	**Specified, unrun**	Deliberate — uncalibrated is worse than none
Production container	**Blocked**	Blocked on authentication (SEC-01), not on the Dockerfile

The defect register reads **"two open, both accepted"** rather than none. That is the honest position: this project distinguishes a cautious failure it chose to carry from a wrong answer it would not.

Evidence: project-context/2.build/{qa,evals,security}.md · docs/demo-feedback.md · week5/assets/

410 turns traced · 168 unit tests · 114 eval assertions · 26 golden items · aamad 0.8.0 clean on define, build and deliver